

OWASP JUICE SHOP – Web Application Vulnerability Assessment

Assessment Environment: Kali Linux

Application Under Assessment: OWASP Juice Shop v20.2.0

Prepared by: Iwuji Daniel

Tools Used:

- OWASP Juice shop
- Kali Linux Terminal
- cURL
- Web Browser
- Git
- VirtualBox

Project Type:

- Web Application Security / Vulnerability Assessment

Assessment Focus:

- Web application reconnaissance
- API security testing
- Authentication and authorization testing
- SQL Injection testing
- Input validation testing
- Error-handling assessment
- Security-header assessment
- CORS configuration assessment
- Access-control testing

- Evidence collection and vulnerability documentation

Assessment Scope:

Local OWASP Juice Shop instance hosted at localhost: *3000*

EXECUTIVE SUMMARY

This project documents a web application security assessment of the intentionally vulnerable OWASP Juice Shop application conducted in a controlled local laboratory environment.

The assessment was performed to develop practical skills in web application reconnaissance, API security testing, authentication testing, input validation, access-control assessment, security-header analysis, and vulnerability documentation.

During testing, a SQL Injection vulnerability resulting in authentication bypass and administrative account access was successfully demonstrated. Additional security observations were identified involving verbose error responses, CORS configuration, and HTTP security-header configuration.

Testing was conducted against an OWASP Juice Shop v20.2.0 instance running locally on Kali Linux. All testing was performed within the controlled laboratory environment for cybersecurity learning and practical vulnerability-assessment experience.

The project demonstrates a structured approach to identifying, validating, documenting, and assessing web application security weaknesses, including the collection of supporting evidence and the development of appropriate remediation recommendations.

ASSESSMENT OBJECTIVE

The primary objectives of this assessment were to:

1. Identify potential security weaknesses within the OWASP Juice Shop web application.
2. Assess the security of application APIs and their responses.
3. Evaluate authentication mechanisms for common web application vulnerabilities.
4. Test application input handling and validation.
5. Assess access-control behaviour for protected resources.
6. Review HTTP security headers and CORS configuration.
7. Identify and validate vulnerabilities using controlled testing techniques.
8. Collect and preserve screenshots and other evidence to support identified findings.
9. Document security findings according to their potential impact and severity.
10. Provide practical remediation recommendations for identified weaknesses.

METHODOLOGY & ASSESSMENT APPROACH

The assessment followed a structured, evidence-based vulnerability assessment methodology designed to identify, validate, and document security weaknesses within the OWASP Juice Shop application.

The assessment was conducted in the following stages:

1. Lab Preparation and Scope Definition

The OWASP Juice Shop application was deployed locally in a controlled Kali Linux laboratory environment. The assessment scope was limited to the locally hosted application to ensure that all testing activities remained within an authorized environment.

2. Application Reconnaissance

Initial reconnaissance was performed to understand the application's accessible pages, endpoints, APIs, responses, and available functionality. HTTP responses and application behaviour were reviewed to establish a baseline before security testing.

3. API Security Testing

Application API endpoints were identified and tested using controlled requests. Responses were examined for authentication requirements, input handling, error behaviour, data exposure, and access-control enforcement.

4. Input Validation and Injection Testing

Controlled test inputs were submitted to application parameters to evaluate how the application handled malformed, unexpected, and potentially malicious input. SQL Injection and XSS-related inputs were tested where appropriate.

5. Authentication and Access-Control Testing

Authentication mechanisms were assessed using invalid credentials and controlled SQL Injection testing. Protected API resources were also tested without authentication to determine whether appropriate access controls were enforced.

6. Security Configuration Assessment

HTTP response headers and CORS behaviour were reviewed to identify security-relevant configuration issues and potential information exposure.

7. Vulnerability Validation

Potential findings were manually validated before being classified as security vulnerabilities. Findings were distinguished from informational observations to avoid overstating the security impact of individual test results.

8. Evidence Collection and Documentation

Screenshots and relevant command-line outputs were captured throughout the assessment to provide evidence supporting the testing process and identified findings.

9. Risk Assessment and Recommendations

Confirmed vulnerabilities and security observations were documented according to their potential impact. Appropriate remediation recommendations were developed to address the identified weaknesses.

ASSESSMENT APPROACH

The assessment followed a reconnaissance → testing → validation → evidence collection → analysis → remediation approach. Testing was performed incrementally, with application responses and observed behaviour used to determine whether additional testing was warranted.

Particular emphasis was placed on validating findings rather than assuming that unusual application behaviour represented a vulnerability.

TOOLS & TECHNOLOGIES USED

Tool / Technology	Purpose
Kali Linux	Primary security testing environment

OWASP Juice Shop v20.2.0	Intentionally vulnerable web application under assessment
cURL	HTTP/API request and response analysis
Web Browser	Application interaction and vulnerability validation
VirtualBox	Local virtualized laboratory environment
Git / GitHub	Project documentation and portfolio management

These tools were used to perform reconnaissance, API testing, authentication testing, input validation testing, security-configuration review, vulnerability validation and evidence collection.

ASSESSMENT ENVIRONMENT & LAB SETUP

The assessment was conducted within a controlled virtual cybersecurity laboratory designed to safely reproduce common web application security vulnerabilities.

Host Environment: Windows 10 Pro

Virtualization Platform: Oracle VirtualBox

Security Testing Environment: Kali Linux

Application Under Assessment: OWASP Juice Shop v20.2.0

Application Address: `http://localhost:3000`

OWASP Juice Shop was hosted locally within the Kali Linux environment and accessed through a web browser and command-line HTTP requests. The application was intentionally selected because it provides a controlled environment for practicing common web application security testing techniques.

All testing activities described in this report were performed against the locally hosted laboratory application. No unauthorized external systems or third-party applications were targeted.

Laboratory Architecture

The testing workflow consisted of:

`Windows 10 Host → VirtualBox → Kali Linux → OWASP Juice Shop → Local Web Browser / cURL`

This isolated configuration provided a controlled environment for performing vulnerability testing while minimizing the risk of affecting external systems.

Application Verification

The Juice Shop instance was successfully verified as operational using an HTTP request to the local application. The server returned HTTP 200 OK, confirming that the application was accessible and responding normally before further testing was conducted.

RECONNAISSANCE & INITIAL DISCOVERY

Initial reconnaissance was conducted to establish the application's baseline behaviour and identify accessible functionality before attempting vulnerability validation.

The OWASP Juice Shop application was confirmed to be running locally on port 3000. An HTTP **HEAD** request to the application returned HTTP 200 OK, confirming that the web application was accessible and responding normally.

The application's product-search functionality was subsequently identified through the **/rest/products/search** endpoint. Controlled requests using normal search terms demonstrated that the endpoint returned structured **JSON** responses containing product information.

For example, a search for **"apple"** returned matching product records, while a search for a non-existent term such as **"xyz123"** returned an empty data set. This established normal application behaviour before input-validation and injection testing.

A direct request to **/api/Products/1** also returned valid product information, while a request for a non-existent product ID returned Not Found. These tests provided a baseline for evaluating API response handling and access-control behaviour.

The application's Score Board was also reviewed to identify available security challenges and track challenge status throughout the assessment.

Initial Reconnaissance Findings

The reconnaissance phase established the following:

- The Juice Shop application was accessible and operational.
- The application exposed multiple REST API endpoints.
- Product information could be retrieved through API requests.
- Normal and non-existent search queries were handled differently as expected.
- Invalid product IDs were handled without exposing a successful resource.
- The Users API required authentication.
- The application returned detailed error information for certain unauthorized requests.
- The Juice Shop Score Board recorded challenge progress during testing.

These observations established the baseline required for the subsequent vulnerability assessment.

AUTHENTICATION SECURITY TESTING

Authentication testing was performed to evaluate whether the Juice Shop login functionality properly validated user-supplied credentials and resisted common authentication-bypass techniques.

➤ Baseline Invalid Login Test

A baseline test was first performed using deliberately invalid credentials:

Email: `jjokach@gmail.com`

Password: `WrongPassword123!`

The application rejected the login attempt and displayed:

“Invalid email or password”

This established that the normal authentication mechanism was functioning and provided a baseline for subsequent security testing.

Result: Authentication correctly rejected invalid credentials.

➤ SQL Injection Authentication Test

A controlled SQL Injection test was subsequently performed against the login functionality using a crafted input in the email field.

The test resulted in a successful authentication response despite the absence of legitimate credentials.

The application subsequently provided access to an administrator account with the following information:

Email: `admin@juice-sh.op`

Username: `SuperUser`

Role: admin

This demonstrated that the application's authentication mechanism could be bypassed through SQL Injection, resulting in unauthorized administrative access.

➤ Security Impact

The successful authentication bypass represents a Critical security finding within the laboratory environment because an attacker capable of exploiting the vulnerability could potentially gain access to an administrative account without possessing valid credentials.

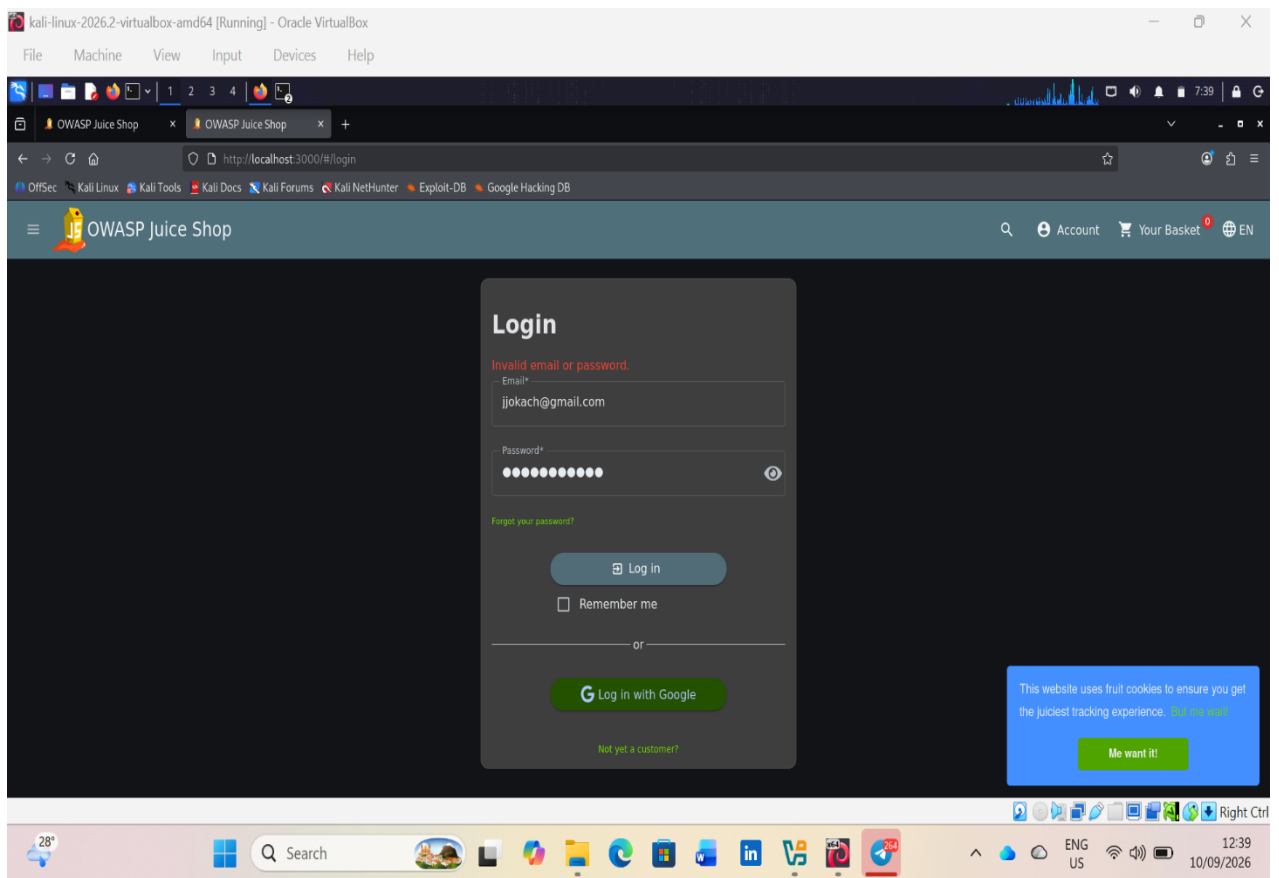
Potential consequences include:

1. Unauthorized administrative access

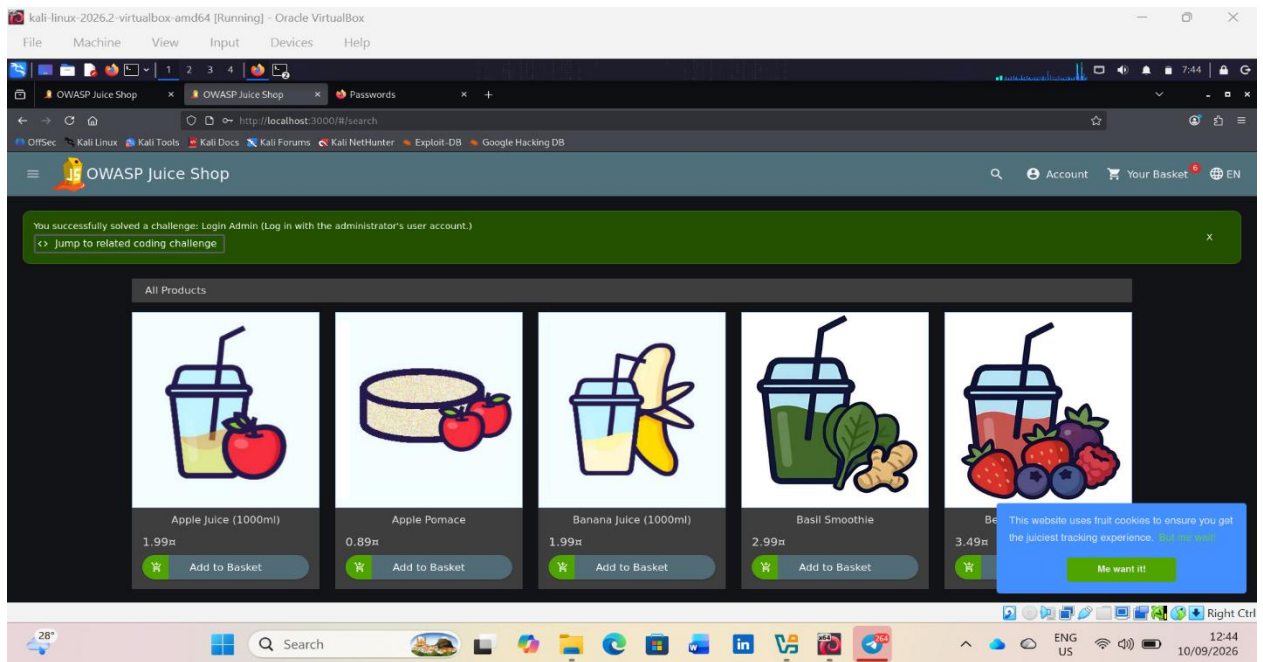
2. Exposure of application data and functionality
3. Unauthorized modification of application resources
4. Potential compromise of other user accounts or application functions
5. Loss of confidentiality, integrity, and availability

➤ **Evidence**

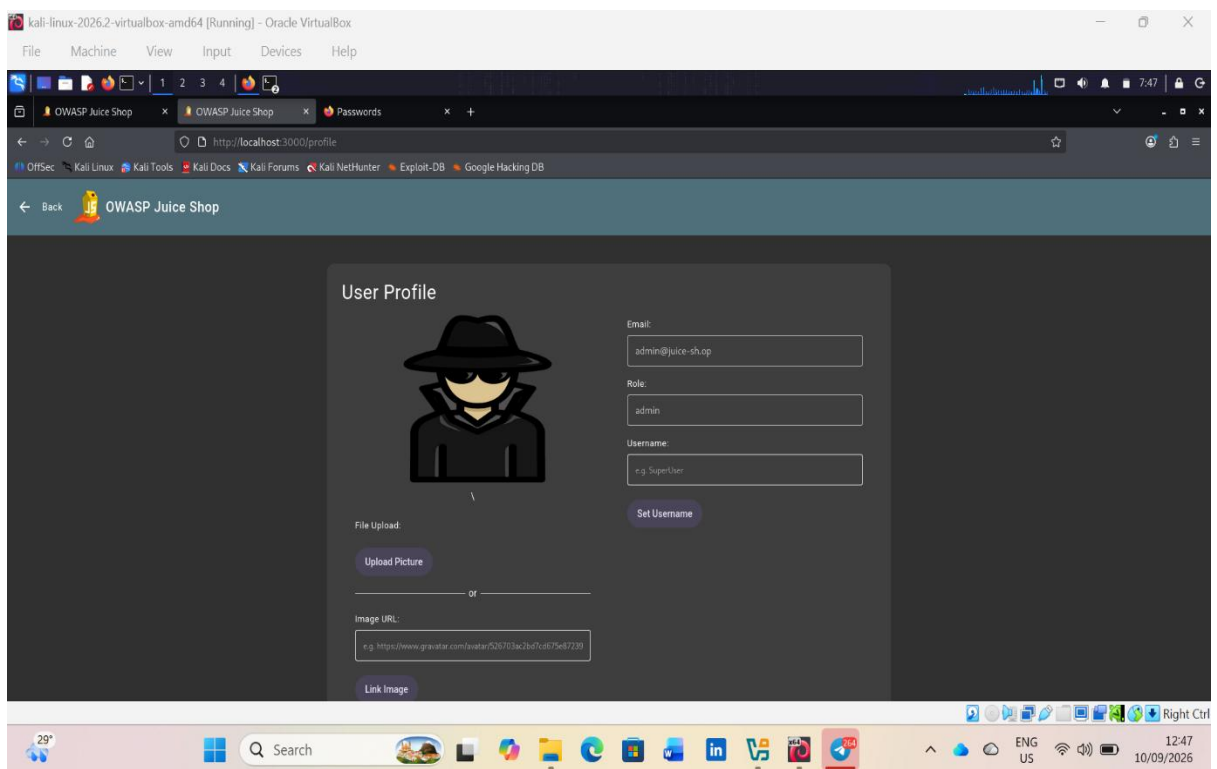
The authentication assessment is supported by the following evidence:



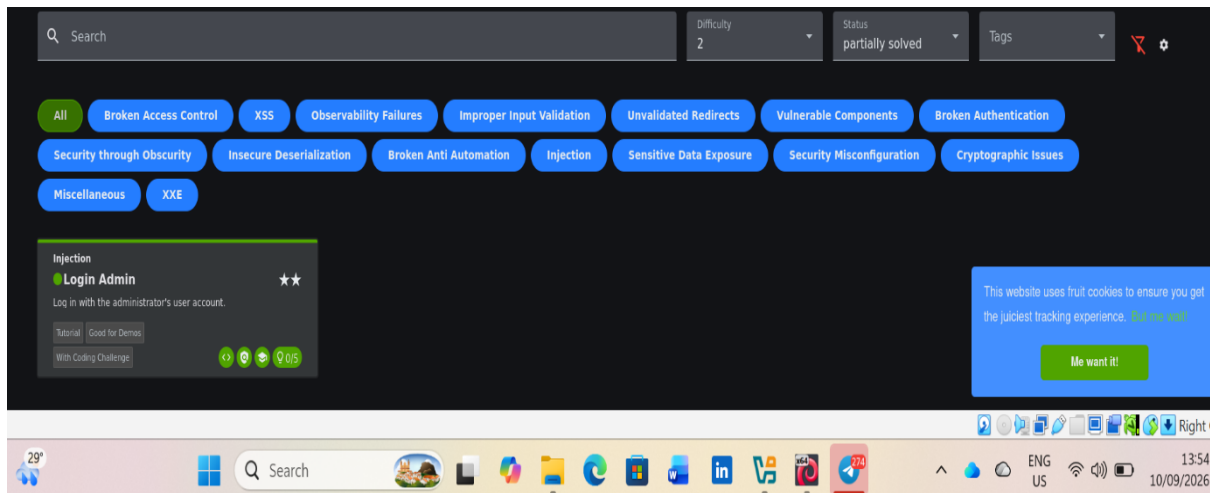
Invalid login baseline — authentication rejected



SQL-injection authentication / administration access result



Successful access to SuperUser administrator account



Login Admin status

IMPROPER ERROR HANDLING & INFORMATION DISCLOSURE

During API security testing, an unauthenticated request was made to the Juice Shop Users API to determine whether protected resources were appropriately restricted.

The application correctly rejected the request with an HTTP 401 Unauthorized response. However, the response also exposed a detailed Express error page containing internal application and framework information.

The response disclosed information including:

Error: UnauthorizedError: No Authorization header was found

Framework: Express 4.22.1

HTTP Status: 401 Unauthorized

Although the authentication control itself functioned correctly, returning a detailed framework error page provides unnecessary information about the application's underlying technology and error-handling implementation.

Security Impact

Verbose error messages can assist attackers during reconnaissance by revealing:

- Application framework information
- Authentication implementation details
- Internal error-handling behaviour
- Potential clues about the application's technology stack

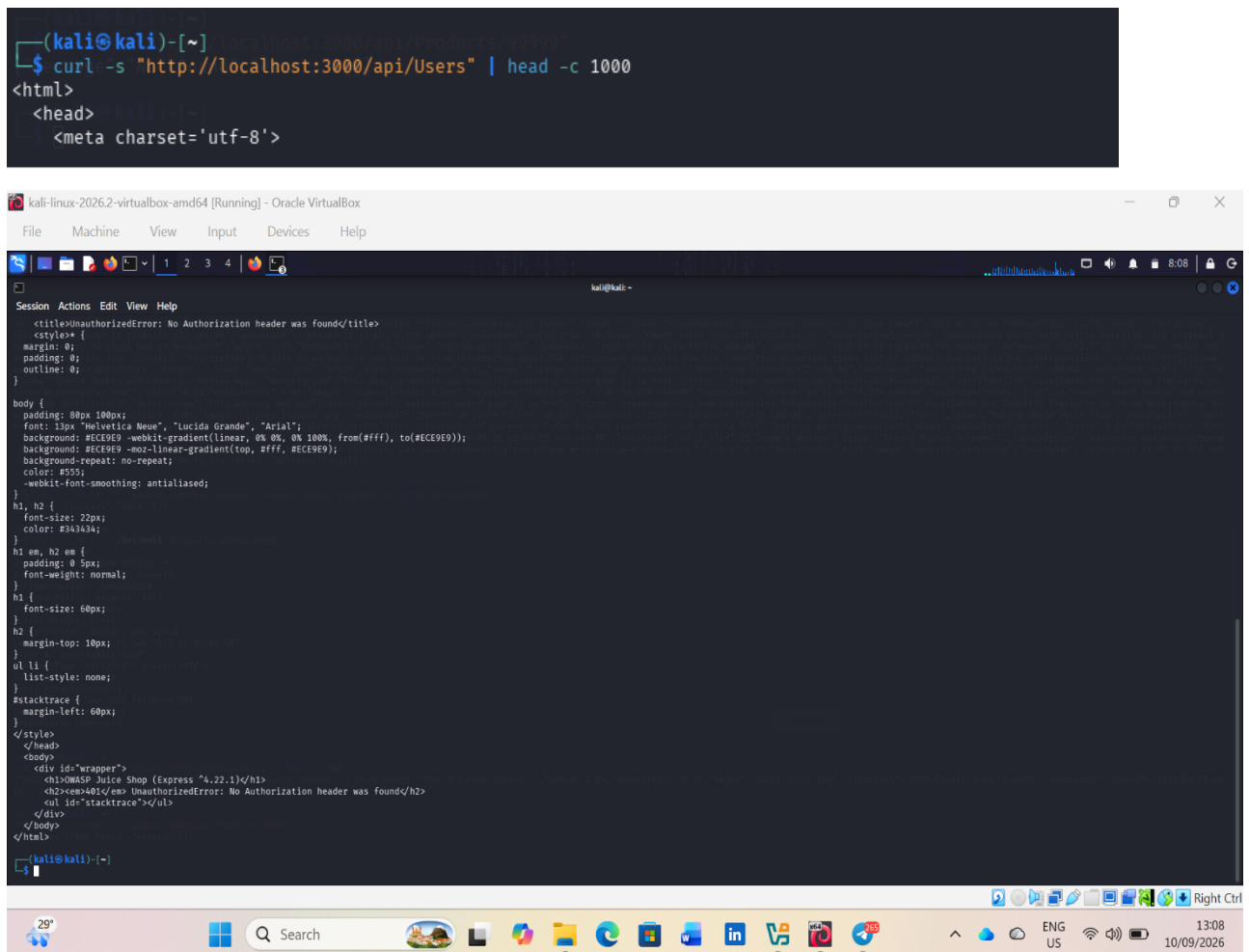
In isolation, the observed information disclosure represents a **Low-to-Medium severity finding**, depending on the production deployment environment and the sensitivity of the information exposed.

Recommendation

The application should implement centralized error handling that returns generic, user-friendly error responses to clients while recording detailed diagnostic information securely in server-side logs.

Production error responses should avoid exposing framework names, stack traces, internal paths, implementation details, or other unnecessary technical information.

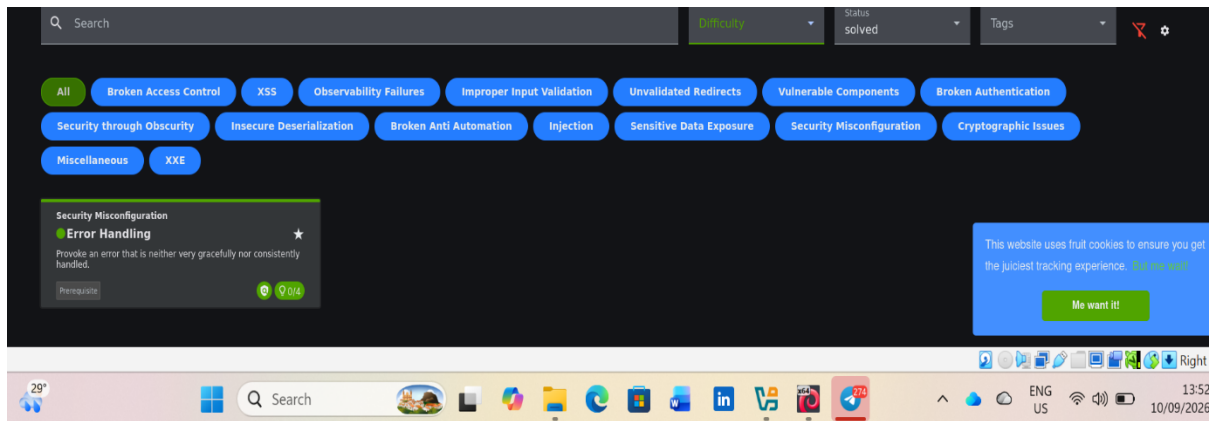
Evidence



The screenshot displays a Kali Linux terminal window at the top, where the command `curl -s "http://localhost:3000/api/Users" | head -c 1000` is executed. The output shows the beginning of an HTML document with a `<meta charset='utf-8'>` tag. Below the terminal is a web browser window titled "kali-linux-2026.2-virtualbox-amd64 [Running] - Oracle VirtualBox". The browser's address bar shows the URL `http://localhost:3000/api/Users`. The page content is an error message: "UnauthorizedError: No Authorization header was found". The browser's developer tools are open, showing the HTML structure. The `<title>` tag contains the error message. The `<h1>` tag contains "OWASP Juice Shop (Express 4.22.1)". The `<h2>` tag contains the error message. The `<ul>` tag contains a stack trace. The browser's status bar at the bottom shows the time as 13:08 on 10/09/2026.

```
(kali@kali)-[~]
$ curl -s "http://localhost:3000/api/Users" | head -c 1000
<html>
  <head>
    <meta charset='utf-8'>
  </head>
  <body>
    <div id='wrapper'>
      <h1>OWASP Juice Shop (Express 4.22.1)</h1>
      <h2>UnauthorizedError: No Authorization header was found</h2>
      <ul id='stacktrace'>
        <li>Error: UnauthorizedError: No Authorization header was found
          at /src/routes/api/users.js:10:11
          at /src/routes/api/users.js:10:11
          at /src/routes/api/users.js:10:11
        </li>
      </ul>
    </div>
  </body>
</html>
```

Unauthenticated User API Test



Error Handling Challenge Solved

API SECURITY & ACCESS-CONTROL ASSESSMENT

API security testing was performed to evaluate how the OWASP Juice Shop application handled normal requests, invalid resources, malformed input, and unauthenticated access to protected endpoints.

➤ Product Search API

The product-search endpoint was tested using both valid and non-existent search terms.

A search for “apple” returned matching product records in JSON format, demonstrating normal API functionality. A search for a non-existent term, “xyz123”, returned an empty data set without generating an application error.

Additional controlled input testing was performed against the search functionality using special characters and script-like input. The application returned an empty data set rather than executing the supplied script-like input.

Assessment: No confirmed XSS vulnerability was established through this test.

➤ Product API Resource Handling

A valid product resource request to `/api/Products/1` returned the expected product information.

A request for a non-existent product identifier returned Not Found, demonstrating that the application did not return a valid resource for the invalid identifier.

Assessment: No unauthorized access to the non-existent resource was demonstrated.

➤ Protected Users API

An unauthenticated request was made to the Users API to determine whether authentication controls were enforced.

The application returned:

`HTTP 401 Unauthorized`

with an error indicating that no Authorization header was supplied.

This demonstrates that authentication was required before accessing the protected Users API.

Assessment: Authentication control was successfully enforced for this endpoint.

➤ **CORS Configuration**

HTTP response headers were reviewed during the assessment. The application returned:

Access-Control-Allow-Origin: *

This configuration permits cross-origin requests at the CORS policy level. However, the tested endpoint exposed publicly accessible product information, and the assessment did not demonstrate unauthorized access to sensitive authenticated data through CORS.

Therefore, this result is documented as a security configuration observation rather than a confirmed exploitable CORS vulnerability.

API Security Assessment Summary

The assessment demonstrated that several API resources were appropriately protected against unauthenticated access, while public product information was accessible through the application's API. The CORS wildcard configuration and verbose error response were identified as areas requiring security review, but neither was classified as a critical exploitable vulnerability based on the evidence collected.

SECURITY HEADERS & CORS CONFIGURATION ASSESSMENT

HTTP response headers were reviewed to identify security-relevant configuration controls and potential information disclosure.

The application returned several defensive security headers, including **X-Content-Type-Options: nosniff** and **X-Frame-Options: SAMEORIGIN**. These controls help reduce risks associated with MIME-type sniffing and unauthorized framing of the application.

The application also returned the following CORS configuration:

Access-Control-Allow-Origin: *

This permits cross-origin requests from any origin at the CORS policy level. However, testing was performed against a publicly accessible product endpoint, and no unauthorized access to sensitive authenticated information through CORS was demonstrated.

The assessment therefore treats the wildcard CORS policy as a configuration observation rather than a confirmed exploitable vulnerability.

The application was also observed to return an application-specific **X-Recruiting** header pointing to the application's jobs route. While this does not constitute a significant

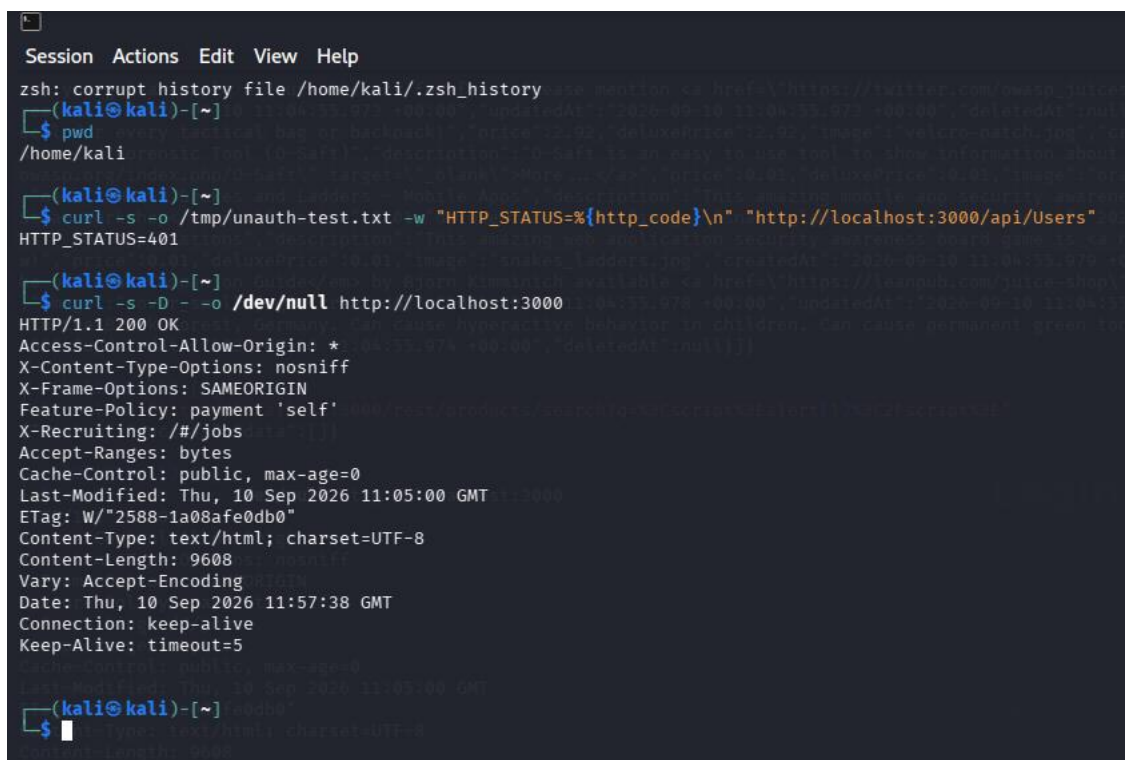
vulnerability by itself, unnecessary custom headers can provide additional information about application functionality.

The assessment also identified the absence of **Content-Security-Policy** and **Strict-Transport-Security** in the observed HTTP response. These headers should be considered for appropriate production deployments, particularly where the application is served over HTTPS. Their absence in the local HTTP laboratory environment was not independently classified as a confirmed vulnerability.

Recommendations

- Restrict CORS to trusted origins where cross-origin access is actually required.
- Avoid exposing unnecessary application-specific response headers.
- Implement an appropriate Content Security Policy for production deployments.
- Implement Strict-Transport-Security when the application is deployed over HTTPS.
- Periodically review HTTP security headers as part of application security testing.

Evidence



```
Session Actions Edit View Help
zsh: corrupt history file /home/kali/.zsh_history
(kali@kali)-[~]
$ pwd
/home/kali
(kali@kali)-[~]
$ curl -s -o /tmp/unauth-test.txt -w "HTTP_STATUS=%{http_code}\n" "http://localhost:3000/api/Users"
HTTP_STATUS=401
(kali@kali)-[~]
$ curl -s -D - -o /dev/null http://localhost:3000
HTTP/1.1 200 OK
Access-Control-Allow-Origin: *
X-Content-Type-Options: nosniff
X-Frame-Options: SAMEORIGIN
Feature-Policy: payment 'self'
X-Recruiting: /#/jobs
Accept-Ranges: bytes
Cache-Control: public, max-age=0
Last-Modified: Thu, 10 Sep 2026 11:05:00 GMT
ETag: W/"2588-1a08afe0db0"
Content-Type: text/html; charset=UTF-8
Content-Length: 9608
Vary: Accept-Encoding
Date: Thu, 10 Sep 2026 11:57:38 GMT
Connection: keep-alive
Keep-Alive: timeout=5
(kali@kali)-[~]
$
```

```
(kali@kali)~$ curl -s -D - -o /dev/null/ "http://localhost:3000/api/Products/1"
HTTP/1.1 200 OK
Access-Control-Allow-Origin: *
X-Content-Type-Options: nosniff
X-Frame-Options: SAMEORIGIN
Feature-Policy: payment 'self'
X-Recruiting: /#/jobs
Content-Type: application/json; charset=utf-8
Content-Length: 257
ETag: W/"101-UI3B0K7pRsGKSxou3JvPzrjgdw"
Vary: Accept-Encoding
Date: Thu, 10 Sep 2026 12:02:09 GMT
Connection: keep-alive
Keep-Alive: timeout=5

{"status":"success","data":{"id":1,"name":"Apple Juice (1000ml)","description":"The all-time classic.","price":1.99,"deluxePrice":0.99,"image":"apple_juice.jpg","createdAt":"2026-09-10T11:04:55.968Z","updatedAt":"2026-09-10T11:04:55.968Z","deletedAt":null}}

(kali@kali)~$ curl -s "http://localhost:3000/api/Products/1" | head -c 300
{"status":"success","data":{"id":1,"name":"Apple Juice (1000ml)","description":"The all-time classic.","price":1.99,"deluxePrice":0.99,"image":"apple_juice.jpg","createdAt":"2026-09-10T11:04:55.968Z","updatedAt":"2026-09-10T11:04:55.968Z","deletedAt":null}}

(kali@kali)~$
```

CORS Product API Assessment

VULNERABILITY FINDINGS & RISK ASSESSMENT

The assessment identified one confirmed high-impact vulnerability and several lower-risk security observations. Findings were classified based on the behaviour demonstrated during testing rather than assumptions about potential exploitation.

Findings Summary

ID	FINDING	SEVERITY	STATUS
F-01	SQL Injection Authentication Bypass	Critical	Confirmed
F-02	Improper Error Handling / Information Disclosure	Low - Medium	Confirmed
F-03	Wildcard CORS Configuration	Informational / Low	Observation
F-04	Security Header Configuration Gaps	Informational / Low	Observation

F-01— SQL Injection / Authentication Bypass

Severity: Critical

Status: Confirmed

Affected Components: Login functionality

Description: A controlled SQL Injection payload submitted through the application's login functionality resulted in successful authentication without legitimate credentials.

The application subsequently provided access to the **SuperUser** account with an administrator role.

Impact: Successful exploitation could allow an attacker to bypass authentication and obtain unauthorized administrative access, potentially compromising application confidentiality, integrity, and availability

Recommendation: Implement parameterized queries/prepared statements for all database operations. User-supplied input should never be directly incorporated into SQL queries. Additional recommendations include centralized input validation, secure authentication controls, least-privilege database permissions, and security testing of all authentication-related endpoints.

F-02— Improper Error Handling / Information Disclosure

Severity: Low - Medium

Status: Confirmed

Affected Component: Users API / Error handling

Description: An unauthenticated request to the Users API was correctly rejected with HTTP 401 Unauthorized. However, the response exposed a detailed Express error page containing framework and error information.

Impact: Verbose error responses can provide attackers with useful information about the application's technology stack and implementation, potentially assisting reconnaissance and subsequent attack development.

Recommendation: Implement centralized error handling and return generic client-facing error messages. Detailed diagnostic information should be restricted to secure sever-side logs.

F-03— Wildcard CORS Configuration

Severity: Informational / Low

Status: Security observation

Description: The application returned **Access-Control-Allow-Origin: *** for the tested product API endpoint.

Impact: A wildcard CORS policy can increase exposure when applied to sensitive endpoints. However, the tested endpoint exposed public product information, and no unauthorized access to sensitive authenticated information through CORS was demonstrated.

Recommendation: Restrict allowed origins to trusted domains where cross-origin access is required.

F-04— Security Header Configuration Gaps

Severity: Informational / Low

Status: Security observation

Description: The application returned several useful security headers, including `X-Content-Type-Options` and `X-Frame-Options`. However, `Content-Security-Policy` and `Strict-Transport-Security` were not observed in the tested HTTP response.

Impact: The absence of additional defensive headers may reduce the application's protection against certain classes of client-side attacks in an appropriately configured production environment.

Recommendation: Implement a suitable Content Security Policy and enable HSTS when the application is deployed over HTTPS. Security headers should be reviewed according to the application's actual production architecture.

REMEDIATION & SECURITY RECOMMENDATIONS

The following recommendations are provided based on the vulnerabilities and security observations identified during the assessment.

1.Prevent SQL Injection

- Use parameterized queries or prepared statements for all database operations.
- Never concatenate or directly embed user-supplied input into SQL queries.
- Apply server-side input validation to authentication and application parameters.
- Use a database account with only the permissions required by the application.
- Include automated security testing for SQL Injection in the development lifecycle.

2.Strengthen Authentication Controls

- Ensure authentication decisions are performed using secure server-side logic.
- Implement secure password hashing and credential-management practices.
- Apply appropriate rate limiting and account-lockout protections against repeated authentication attempts.

- Monitor authentication failures and suspicious login activity.
- Regularly test authentication endpoints for bypass techniques.

3.Improve Error Handling

- Replace verbose client-facing error messages with generic responses.
- Prevent framework names, implementation details, stack traces, and internal information from being exposed to users.
- Store detailed diagnostic information securely in server-side logs.
- Implement centralized application error handling.

4.Review CORS Configuration

- Replace wildcard CORS policies with an explicit allowlist of trusted origins where practical.
- Avoid allowing cross-origin access to sensitive authenticated resources unless specifically required.
- Review whether credentials or authentication tokens are permitted in cross-origin requests.
- Regularly test CORS policies as part of application security assessments.

5.Strengthen HTTP Security Headers

- Implement an appropriate **Content-Security-Policy**.
- Enable **Strict-Transport-Security** when the application is deployed over HTTPS.
- Continue using defensive headers such as **X-Content-Type-Options** and **X-Frame-Options**.
- Periodically review security headers against the application's production requirements.

6.Security Monitoring and Logging

- Monitor authentication failures and unusual authentication patterns.
- Log suspicious input and application errors securely.
- Monitor administrative account activity.
- Establish alerting for repeated SQL Injection attempts and other malicious input patterns.
- Protect logs from unauthorized modification or deletion.

Overall Recommendation

The highest priority should be given to eliminating the SQL Injection vulnerability because the assessment demonstrated that it could result in authentication bypass and administrative

account access. Error handling, CORS configuration, and security-header improvements should then be addressed as part of a broader application-security hardening program.

CONCLUSION & KEY TAKEAWAYS

The OWASP Juice Shop web application security assessment successfully demonstrated the practical process of identifying, validating, documenting, and assessing common web application security weaknesses within a controlled laboratory environment.

The most significant finding was a SQL Injection vulnerability affecting the authentication mechanism, which resulted in successful authentication as the SuperUser administrator account. This demonstrated the potential impact of inadequate input handling and insecure database interaction on authentication security.

The assessment also identified improper error handling and information disclosure, where a protected API correctly rejected an unauthenticated request but returned detailed framework-related error information. Additional observations were made regarding wildcard CORS configuration and HTTP security-header configuration.

Importantly, the assessment also demonstrated several security controls functioning as intended. Invalid credentials were rejected, protected API resources returned HTTP 401 Unauthorized when accessed without authentication, and invalid product resources were handled appropriately.

Overall, the project strengthened practical skills in:

- Web application reconnaissance
- REST API security testing
- Authentication and access-control testing
- SQL Injection identification and validation
- Input validation assessment
- HTTP security-header analysis
- CORS configuration review
- Error-handling assessment
- Evidence collection
- Vulnerability severity assessment
- Security remediation planning

The project reinforces the importance of validating security findings with reproducible evidence and distinguishing confirmed vulnerabilities from security observations and correctly functioning controls.

From a defensive security perspective, the findings highlight the importance of parameterized database queries, secure authentication mechanisms, centralized error handling, restrictive CORS policies, appropriate HTTP security headers, and continuous security testing throughout the application development lifecycle.

This assessment was performed exclusively against the intentionally vulnerable OWASP Juice Shop application within an authorized local laboratory environment.